If statements and latches

Entry 158 · Always_if2 · 7 September 2026

Your solution works because both outputs now receive a value for every input combination. When the CPU cools down, the shutdown signal returns to 0. When you arrive, the driving signal returns to 0 even if there is fuel left.

Your working solution

This keeps the logic from your open editor. The formatting is tidier, and the one-bit constants are written explicitly.

```
module top_module (
    input cpu_overheated,
    output reg shut_off_computer,
    input arrived,
    input gas_tank_empty,
    output reg keep_driving
);
    always @(*) begin
        if (cpu_overheated)
            shut_off_computer = 1'b1;
        else
            shut_off_computer = 1'b0;
    end

    always @(*) begin
        if (~arrived)
            keep_driving = ~gas_tank_empty;
        else
            keep_driving = 1'b0;
    end
endmodule
```

Read the circuit from the assignments

shut_off_computer follows **cpu_overheated**. **keep_driving** is 1 only while you have not arrived and the tank is not empty. Neither output needs a previous value or a clock.

Pages 2–3 explain the missing paths, timing and alternative coding styles. Pages 4–5 answer the cleanup question in your entry 155 sequence-recognizer code.

What the missing else changes

The original driving block

```
always @(*) begin
    if (~arrived)
        keep_driving = ~gas_tank_empty;
end
```

When arrived is 0, the block assigns keep_driving. When arrived becomes 1, the block runs again, but the if body is skipped. No assignment executes. Verilog keeps the previous value of the variable; it does not insert a zero for you.

That behavior needs storage. While arrived is 0, a latch is transparent and its output follows the inverted tank input. While arrived is 1, it holds the last value. The sensitivity list tells the simulator when to evaluate the block. It does not guarantee a combinational circuit.

Event	arrived	empty	Original output	Fixed output
Driving with fuel	0	0	1	1
Reach destination	1	0	1 (held)	0
Tank becomes empty after arrival	1	1	1 (still held)	0
Leave with empty tank	0	1	0	0
Refuel before arrival	0	0	1	1

Why the shutdown block also needs the else

```
if (cpu_overheated)
    shut_off_computer = 1'b1;
// No assignment when cpu_overheated is 0.
```

Once this code writes 1, a later change back to cpu_overheated=0 leaves that 1 unchanged. Before the first assignment, RTL simulation can show X because the variable has no known initial value. The requested circuit must instead produce 0 whenever the CPU is cool.

This incomplete block describes holding behavior, but a synthesis tool may optimize this particular constant-data case into constant logic. Do not depend on seeing a physical latch in the final netlist to detect the mistake. The missing output value is already wrong at the RTL level.

Other ways to write the same rule

Assign a default, then override it

```
always @(*) begin
    shut_off_computer = 1'b0;
    keep_driving = 1'b0;
    if (cpu_overheated)
        shut_off_computer = 1'b1;
    if (!arrived)
        keep_driving = !gas_tank_empty;
end
```

Each evaluation first assigns both defaults. A true condition then replaces its output value within the same evaluation. These blocking assignments describe combinational logic; they do not use two clock cycles. Use this as a replacement for the earlier blocks so each output still has one driver.

The Boolean form

```
assign shut_off_computer = cpu_overheated;
assign keep_driving = !arrived && !gas_tank_empty;
```

For this Verilog-2001 version, declare the continuously driven outputs as **output** or **output wire**, and remove the procedural blocks. This compact form has the same truth table for known 0/1 inputs. With X or Z inputs, direct expressions and an if/else can propagate unknowns differently.

arrived	gas_tank_empty	keep_driving
0	0	1
0	1	0
1	0	0
1	1	0

Details that are easy to mix up

reg does not mean flip-flop. Here it permits assignment inside an always block. Complete combinational assignments infer logic; an incomplete combinational path can infer a latch; a rising-edge block describes edge-triggered storage.

Use = here. Blocking assignment is the usual choice for combinational code. Use <= for the stored state in the clocked FSM on page 4. Changing the assignment operator does not repair a missing path.

begin/end groups statements. One statement after an if or else does not require it. If you add a second assignment to a branch, group both. The else supplies the missing behavior; begin/end alone does not.

~ and !: ~ inverts each bit; ! asks whether the whole value is zero. For the one-bit, known inputs here they agree. On a vector, they differ, so choose the operator for the meaning you want.

Source: HDLBits — Always_if2. The open page showed Status: Success! for your corrected code.

A cleaner 1101 recognizer

Entry 155 · Exams/review2015_fsmseq

Your code asks: “any better way to code this solution?” The state transitions are already right. Keep the same five states, give them names that describe the matched prefix, and remove the unused s5, the empty if(state==s4) branch and the commented-out output assignments.

```
module top_module (
    input clk, input reset, input data,
    output start_shifting
);
    localparam IDLE  = 3'd0, SEEN1 = 3'd1,
               SEEN11 = 3'd2, SEEN110 = 3'd3,
               FOUND = 3'd4;
    reg [2:0] state, next_state;

    always @(posedge clk) begin
        if (reset)
            state <= IDLE;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            IDLE:    next_state = data ? SEEN1 : IDLE;
            SEEN1:   next_state = data ? SEEN11 : IDLE;
            SEEN11:  next_state = data ? SEEN11 : SEEN110;
            SEEN110: next_state = data ? FOUND : IDLE;
            FOUND:   next_state = FOUND;
            default: next_state = IDLE;
        endcase
    end
    assign start_shifting = (state == FOUND);
endmodule
```

Each case branch assigns next_state, including default, so the earlier next_state=state default is optional here. localparam keeps internal state codes from being overridden as module parameters. The existing three-bit encoding is sufficient for five states.

start_shifting is decoded from state. After the edge that accepts the final 1, state becomes FOUND and the output rises without waiting for another rising edge. Reset is synchronous: it takes effect when a rising edge samples reset=1.

Why these five states work

State	What has matched	Next on 0	Next on 1
IDLE	No useful suffix	IDLE	SEEN1
SEEN1	1	IDLE	SEEN11
SEEN11	11	SEEN110	SEEN11
SEEN110	110	IDLE	FOUND
FOUND	Complete 1101	FOUND	FOUND

The SEEN11 self-loop matters

After 111, the final two bits are still 11. Staying in SEEN11 lets 11101 match using the last four bits. Moving back to IDLE on the extra 1 would lose that useful suffix.

Rising edge	Sampled input	State after edge	start_shifting
Reset	reset=1	IDLE	0
1	1	SEEN1	0
2	1	SEEN11	0
3	1	SEEN11	0
4	0	SEEN110	0
5	1	FOUND	1
6	0	FOUND	1

Holding FOUND is intentional

This exercise asks for start_shifting to stay high until reset. FOUND therefore transitions to itself for either input. The clocked state register provides that memory. In Always_if2, a combinational output accidentally held a value because an assignment was missing. Here every next_state path is assigned, and the stored state changes only at the clock edge.

Would fewer lines make a better circuit?

A shift-register detector can also recognize 1101, but it needs a way to keep the result high after detection. Your FSM already records the useful input suffix and the permanent-found condition. The cleanup above makes that behavior easier to read without changing the design.

Checks used for these notes

The local Always_if2 test covers all eight binary input combinations and a drive/arrival sequence that exposes the original held output. The cleaned FSM is checked against a four-bit sliding-window reference over every 12-bit input stream, with sticky detection and synchronous reset checks.

Source: HDLBits — 1101 sequence recognizer. Entries 155–158 each showed Status: Success! in their open result panels. The local tests check the examples printed here.